

PAQI

Pearls AQI Predictor

A Machine Learning System for 3-Day Air Quality Forecasting

By Syed Usman Ali

Project Report

Prepared for 10Pearls

Executive Summary

PAQI (Pearls AQI Predictor) is an end-to-end machine learning system that forecasts air quality (AQI) 1 to 3 days in advance for five major Pakistani cities: Karachi, Lahore, Islamabad, Peshawar, and Quetta. The system covers the full pipeline from data collection through to a live, public-facing dashboard, and was built and iterated on over several development phases.

The system pulls hourly air quality and weather data from Open-Meteo, engineers time-series features (lags, rolling averages, seasonal indicators), trains and compares several model families, and serves the best-performing model through a Streamlit dashboard. The best model is validated both on a scattered cross-seasonal split and, more strictly, on a 90-day window held out entirely from training.

This report documents the system architecture, data methodology, model development and validation process, the live dashboard, the infrastructure the system runs on, and the operational issues encountered and resolved during development.

Table of Contents

Executive Summary	2
Table of Contents	3
1. Introduction.....	4
1.1 Problem Statement	4
1.2 Objectives	4
2. System Architecture	4
3. Data Sources and Methodology	5
3.1 Data Source Consistency	5
4. Feature Engineering	6
5. Model Development and Selection	6
6. Validation Methodology.....	6
6.1 Scattered Cross-Seasonal Split	6
6.2 Genuine Recent Holdout (Stricter Test)	7
7. Dashboard	7
8. Infrastructure and Deployment.....	9
9. Challenges Encountered and Resolutions	11
9.1 Data Source Consistency	11
9.2 Forecast Data Contamination in Backfill	11
9.3 Scheduling Reliability.....	11
9.4 Third-Party API Reliability.....	11
9.5 Resource Constraints.....	11
9.6 Pipeline Runtime	11
9.7 Backend Cost Overrun and Migration to Supabase	12
10. Limitations and Future Work.....	13
11. Conclusion	13
References.....	14

1. Introduction

1.1 Problem Statement

Air pollution is a serious and growing public health concern in Pakistani cities, with AQI levels frequently reaching Unhealthy or Very Unhealthy categories, particularly during the winter smog season. Residents currently have access to current air quality readings through various apps, but reliable, localized forecasting - which would let people plan ahead rather than only react - is far less available.

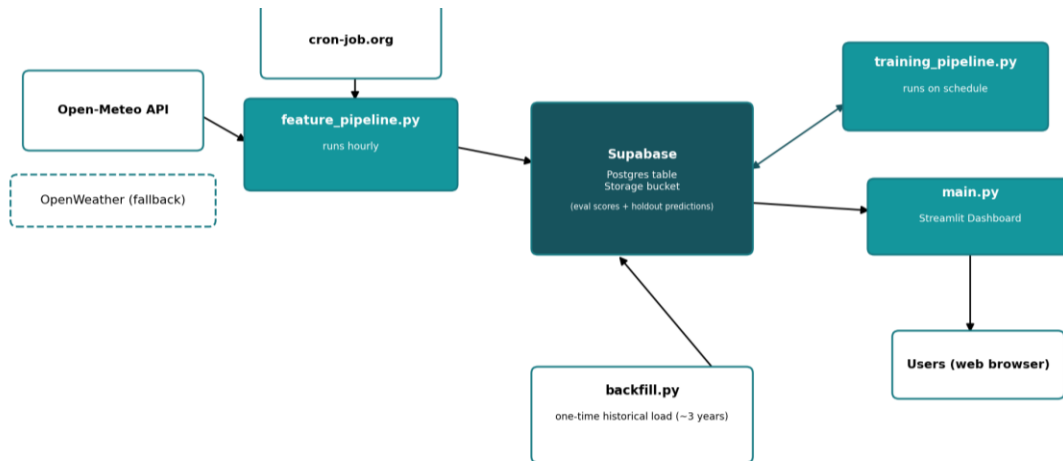
1.2 Objectives

- Build a reliable pipeline to collect and store historical and ongoing air quality and weather data for multiple Pakistani cities.
- Engineer features that capture seasonal, weather-driven, and short-term temporal patterns in AQI.
- Train and rigorously validate a forecasting model for 1, 2, and 3 days ahead.
- Deploy the model behind a public, usable dashboard showing current conditions, forecasts, and model explainability.
- Ensure the system runs autonomously on a schedule, with data and model refreshed without manual intervention.

2. System Architecture

The system is built as four cooperating components, all operating on a shared Supabase backend (a Postgres table for feature data and a Storage bucket for model artifacts):

- `backfill.py` - a one-time (or occasionally re-run) script that populates the feature table with roughly 3 years of hourly historical data.
- `feature_pipeline.py` - runs hourly via GitHub Actions, fetching the latest hour's readings and appending them to the feature table.
- `training_pipeline.py` - runs on a schedule via GitHub Actions, trains and evaluates several model families, and publishes the best one to the model registry (Supabase Storage).
- `main.py` - the public Streamlit dashboard, which loads the latest model and data from Supabase on every run.



3. Data Sources and Methodology

All air quality and weather data - for both historical backfill and ongoing hourly updates - is sourced from Open-Meteo's Air Quality and Weather APIs. AQI values use the official US EPA methodology, evaluated across all six pollutants (PM2.5, PM10, CO, NO2, SO2, O3), not a simplified PM-only approximation.

Each city is represented by five geographic points (its primary point plus four other district points), which are used both to compute a district-level spread feature and to give the model a sense of intra-city variation in air quality.

3.1 Data Source Consistency

An early version of the pipeline computed AQI differently depending on data source (Open-Meteo's full 6-pollutant method for most data, versus a PM-only approximation for OpenWeather-sourced rows), and used two entirely different providers for weather variables between the historical backfill and the live hourly pipeline. This was identified as a data consistency risk during development and corrected: the live pipeline now sources both air quality and weather from Open-Meteo by default, matching the training data's methodology.

A last-resort fallback to OpenWeather remains in place for the rare cases where Open-Meteo is unreachable (occasional throttling has been observed from GitHub Actions' shared IP ranges). Every row produced this way is explicitly flagged, so fallback usage is auditable rather than silently blended into the dataset.

4. Feature Engineering

Hourly readings are aggregated to daily granularity per city, then engineered into the following feature groups:

- Lag features: AQI from 1, 2, 3, and 7 days prior.
- Rolling statistics: 3-, 7-, and 14-day rolling mean AQI, and 7-day rolling standard deviation.
- Rate of change: day-over-day AQI change.
- Seasonal indicators: winter smog season, dust season, monsoon season flags, plus cyclical month encoding (sine/cosine).
- District-level statistics: median, max, min, and spread of AQI across each city's five monitored points.
- Weather features: temperature, humidity, pressure, wind speed and direction (cyclically encoded), and precipitation.
- An intra-day volatility feature (hourly AQI standard deviation and daily peak) capturing how much a city's air quality swings within a single day.

5. Model Development and Selection

Several model families were trained and compared using a randomized hyperparameter search: Random Forest, Histogram-based Gradient Boosting, XGBoost, LightGBM, CatBoost, Ridge Regression, a small feedforward neural network, and a stacked ensemble combining the tree-based models. All models predict log-transformed AQI simultaneously for the 1, 2, and 3-day horizons.

6. Validation Methodology

Two independent evaluations are used, each compared against a naive persistence baseline ("tomorrow's AQI equals today's AQI"):

6.1 Scattered Cross-Seasonal Split

Every sixth day across the entire multi-year dataset is held out as a test set, ensuring the model is evaluated across every season represented in the data, not just one part of the year.

6.2 Genuine Recent Holdout (Stricter Test)

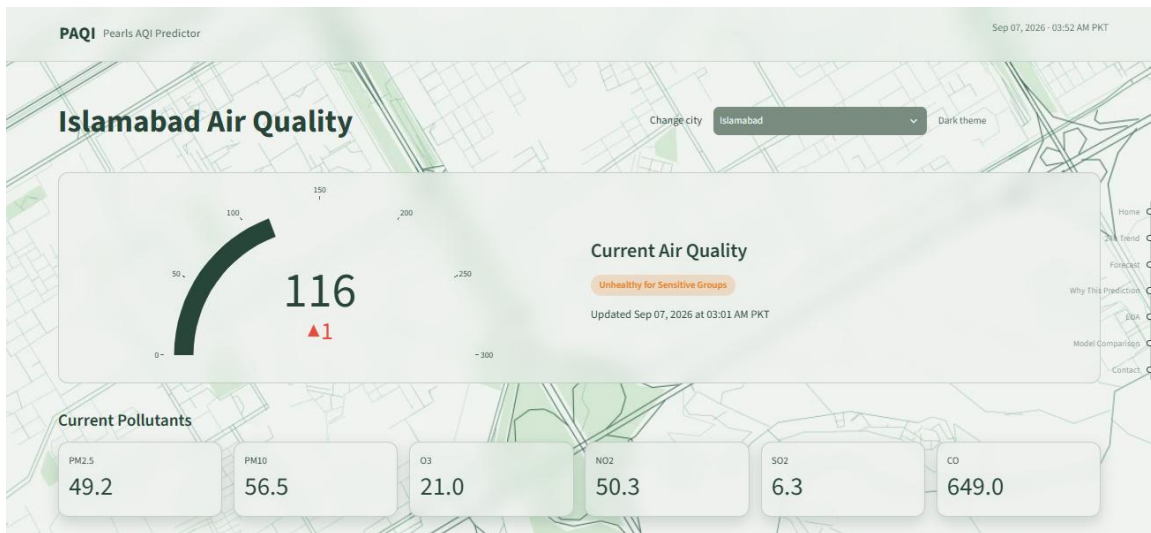
The final 90 days of data are excluded from the training pool entirely before model selection and hyperparameter search take place. The deployed model is then evaluated on this window, which it has never seen in any form. This is a stricter and more realistic test of forward forecasting ability than a scattered split can provide.

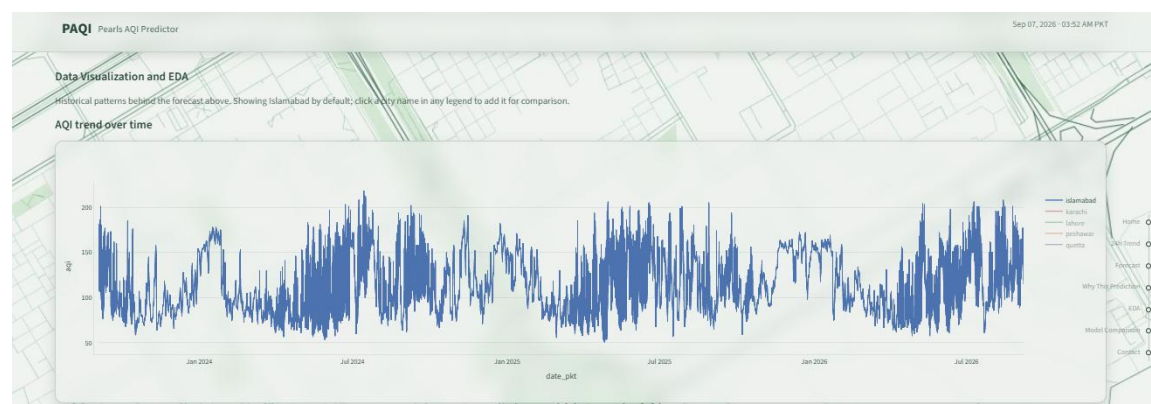
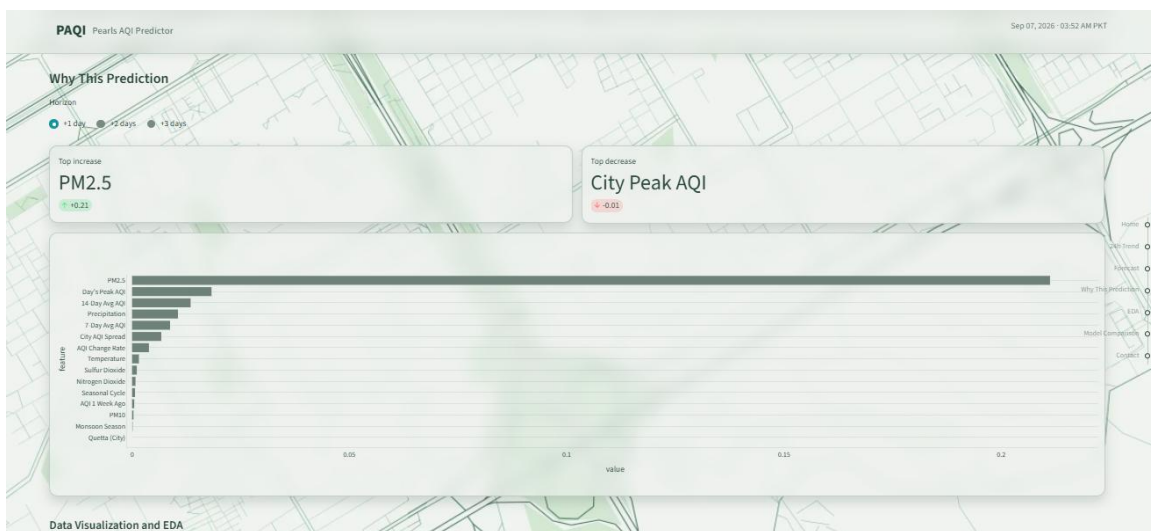
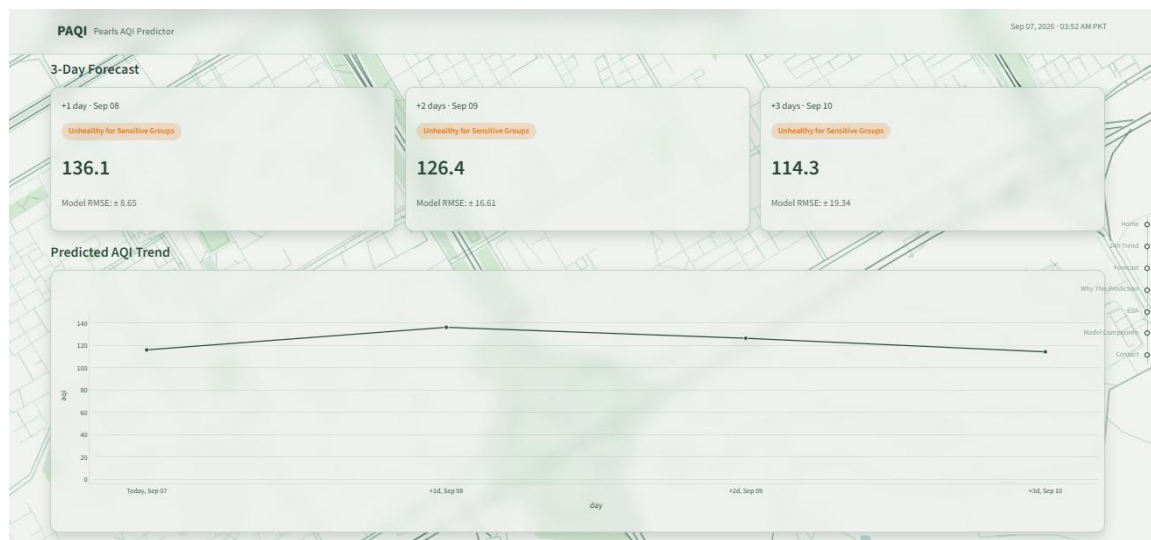
Evaluation	RMSE	MAE	R ²
Every-6th-day split	15.54	11.29	0.835
90-day genuine holdout	18.24	13.10	0.647

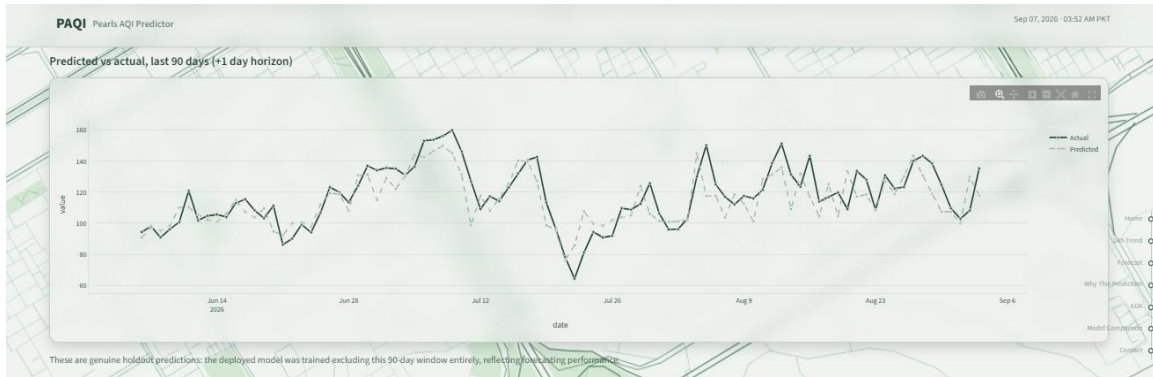
Both evaluations beat the persistence baseline on their respective test sets, and the model and its per-horizon RMSE are loaded directly from the model registry (Supabase Storage) by the dashboard on every run, so the figures shown to users always reflect whichever model is currently deployed rather than a stale, hardcoded value.

7. Dashboard

The Streamlit dashboard presents, for a user-selected city, the current AQI and category, a 3-day forecast with per-horizon model RMSE, a 24-hour AQI trend, current pollutant readings, a SHAP-based explanation of what is driving the forecast, and an exploratory data analysis section covering historical trends, pollutant distributions, correlations, and outliers. The dashboard also includes a footer with project links and contact details for feedback or questions.







Predicted vs actual, last 90 days (+1 day horizon)

PAQI Pearls AQI Predictor

Sep 07, 2026 - 03:52 AM PKT

Model Comparison

Top 3 models from training, evaluated on the same held-out split.

Model	R2	RMSE	MAE
HistGB	0.835	15.54	11.29
XGBoost	0.827	15.92	11.34
RandomForest	0.826	15.97	11.42

Full comparison table

Horizon	Model	RMSE	MAE	R2
Day 1	HistGB	6.43	8.65	0.950
Day 1	XGBoost	6.34	8.61	0.950
Day 1	RandomForest	6.51	8.92	0.947
Day 2	HistGB	12.74	16.61	0.811
Day 2	XGBoost	12.82	16.89	0.804
Day 2	RandomForest	12.90	16.98	0.802
Day 3	HistGB	14.71	19.34	0.744
Day 3	RandomForest	14.86	19.92	0.728
Day 3	XGBoost	14.86	20.03	0.725

Model Comparisons

8. Infrastructure and Deployment

- Supabase - hosts the feature data (a Postgres table) and the model registry artifacts (a Storage bucket): the deployed model, its evaluation scores, and holdout predictions.
- GitHub Actions - runs feature_pipeline.py and training_pipeline.py on schedule.
- Streamlit Community Cloud - hosts the public-facing dashboard.
- cron-job.org - an external scheduler that reliably triggers the hourly feature pipeline via GitHub's workflow_dispatch API, working around GitHub's own best-effort (and occasionally unreliable) scheduled-trigger delivery.

Feature Pipeline Feature Pipeline #483: Manually run by usmanali2318	main	Today at 10:00 PM 17m 34s ...
Feature Pipeline Feature Pipeline #481: Manually run by usmanali2318	main	Today at 9:00 PM 14m 3s ...
Feature Pipeline Feature Pipeline #480: Manually run by usmanali2318	main	Today at 8:00 PM 10m 30s ...
Feature Pipeline Feature Pipeline #479: Manually run by usmanali2318	main	Today at 7:00 PM 17m 4s ...
Feature Pipeline Feature Pipeline #478: Manually run by usmanali2318	main	Today at 6:00 PM 17m 5s ...
Feature Pipeline Feature Pipeline #477: Manually run by usmanali2318	main	Today at 5:00 PM 14m 0s ...

Training Pipeline Training Pipeline #52: Scheduled	main	Today at 2:50 PM 8m 30s ...
Training Pipeline Training Pipeline #51: Scheduled	main	Aug 28, 8:07 PM GMT+5 10m 39s ...
Training Pipeline Training Pipeline #50: Scheduled	main	Aug 27, 6:56 PM GMT+5 10m 33s ...
Training Pipeline Training Pipeline #49: Scheduled	main	Aug 26, 8:55 AM GMT+5 9m 54s ...
Training Pipeline Training Pipeline #48: Scheduled	main	Aug 25, 8:51 AM GMT+5 9m 41s ...
Training Pipeline Training Pipeline #47: Scheduled	main	Aug 24, 8:56 AM GMT+5 9m 14s ...

9. Challenges Encountered and Resolutions

9.1 Data Source Consistency

Described in Section 3.1: mixed AQI methodology and mixed weather sources between training and live data were identified and corrected.

9.2 Forecast Data Contamination in Backfill

Open-Meteo's air quality endpoint was found to return forecast-model data beyond the requested date range rather than strictly historical values. This was corrected by explicitly clipping backfilled data to the current timestamp at fetch time, preventing forecast values from being treated as historical ground truth during training.

9.3 Scheduling Reliability

GitHub Actions' native cron scheduling does not guarantee timely (or even eventual) delivery of scheduled triggers, particularly at common trigger times. This was mitigated first by offsetting the trigger away from the top of the hour, and ultimately resolved by moving the real schedule to an external cron service that calls GitHub's `workflow_dispatch` API directly, with GitHub's native schedule retained only as an infrequent backup.

9.4 Third-Party API Reliability

Open-Meteo occasionally throttles or times out requests from GitHub Actions' shared IP ranges. The pipeline was made resilient to this at two levels: retries with backoff on individual requests, isolation of failures per city so one city's failure does not discard data for the other four, and a clearly-flagged last-resort fallback to OpenWeather so a bad hour never means zero data collected.

9.5 Resource Constraints

The original feature store/model registry backend's free tier included a limited monthly compute budget, which was exceeded during a period of frequent full feature-store rebuilds. This was mitigated by avoiding unnecessary full backfills and by evaluating (and deliberately rejecting) a rolling-deletion scheme, since the actual bottleneck was compute rather than storage, and added delete operations would have increased compute load rather than reduced it. This constraint recurred later at greater severity and is addressed fully in Section 9.7.

9.6 Pipeline Runtime

The hourly pipeline's per-city data collection made its API calls sequentially (7 calls per city, one of them a redundant duplicate fetch of the primary point), which compounded with API retries into run times exceeding 25 minutes. This was resolved by deduplicating the redundant fetch and parallelizing both the per-city district calls and the five cities themselves using a thread pool, substantially reducing total run time.

9.7 Backend Cost Overrun and Migration to Supabase

The original backend's free tier billed per data-operation rather than by data volume or by a flat monthly allowance. Under regular use - the hourly pipeline writing new rows, the daily training run reading the full historical dataset, and live dashboard visits each triggering their own reads - this consumed the entire monthly free-tier allowance in roughly a day, well before the reporting period it was meant to cover.

The root cause was isolated through direct measurement rather than guesswork: every call to the backend was logged with a timestamp, and the hourly pipeline was disabled for a controlled period to observe cost behavior in isolation. This confirmed that per-operation billing, not data size, was driving the overrun, and that the dashboard's own read frequency was a significant contributor alongside the scheduled pipelines.

Because reducing data frequency was not an acceptable option (hourly resolution was a firm requirement), the system was migrated to Supabase, which uses a flat monthly resource allowance rather than per-operation billing. The feature store became a Postgres table and the model registry became a Storage bucket. Every existing optimization carried over unchanged: incremental reads seeded from a cached snapshot instead of re-reading full history on each run, and paginated bulk reads for the one-time historical backfill. The migration was performed incrementally with the previous backend's code retained (disabled, not deleted) until each component was verified working end-to-end on Supabase, allowing a fast rollback at every step had one been needed. None was.

10. Limitations and Future Work

- Forecast accuracy degrades at longer horizons (+2 and +3 days) relative to +1 day, reflecting the inherent difficulty of longer-range air quality forecasting and the model's heavy reliance on same-day PM2.5 readings.
- The live “current AQI” reflects the most recent hourly pipeline run rather than a live-at-page-load reading.
- Supabase's free tier is a flat monthly allowance rather than per-operation billing, which resolved the cost issue described in Section 9.8, but it still imposes storage and bandwidth ceilings; continued growth in historical data volume will eventually require a paid tier or an archival strategy for older rows.
- The fallback data path, while rare and flagged, is a residual and disclosed limitation on full data source consistency.

11. Conclusion

PAQI demonstrates a complete, operational machine learning system for air quality forecasting, covering data engineering, model development, rigorous validation, and public deployment. Development surfaced and resolved several real-world issues common to production ML systems - data source consistency, scheduling reliability, backend cost management, and third-party API resilience - each addressed with a documented, verifiable fix rather than a workaround that merely hid the symptom.

References

1. GitHub Docs, "Events that trigger workflows – schedule," [Events that trigger workflows - GitHub Docs](#)
2. Open-Meteo, "Air Quality API," [Air Quality API | Open-Meteo.com](#)
3. Open-meteo, GitHub Issue #1651 (Open-Meteo IP/timeout reports), [Timeout on API · Issue #1651 · open-meteo](#)
4. Open-meteo, GitHub Issue #1227 (forecast data returned past requested end_date), [Historical Air Quality Data? · Issue #1227 · open-meteo](#)
5. US EPA / AirNow. Technical basis for the Air Quality Index (AQI) breakpoints and category thresholds, [AQI Basics | AirNow.gov](#)
6. Supabase Storage Size Usage. [Manage Storage size usage | Supabase Docs](#)